

Parallel and Distributed Computing

Dr. Shajulin Benedict
shajulin@iiitkottayam.ac.in

Syllabus

- Architectures – Multi-core and Many-core architectures, Accelerators (SIMD units -Vectorization, GPUs), Goals of parallel systems.
-
- Applications – Scientific applications, Characteristics, requirements, regular grid applications, irregular applications, data dependence, parallelization process.
-
- Parallel Programming on Shared Memory – OpenMP (C++ languages), Execution Model, Shared and private data, Directives, Barriers, Sections, Run-Time library functions, scheduling strategies, Scalability study, OpenMP for accelerator programming.
-
- Parallel Programming on Distributed Memory – MPI, Collective operations, Non-Blocking, Collectives, Process topologies, Parallel I/O, Single sided communications.
-
- Performance Tools – Concepts, Event-Model execution, profiling, tracing, types of profiling, profiling tools – Scalasca, Score-P, MPI-P, EnergyAnalyzer, Tracing tools, Autotuning – Periscope Tuning Framework.

Applications – 2 Broad classifications

- Scientific Applications

- Computational problems (incl. ML algorithms)
 - N-body simulation
 - In physics, the **n-body problem** is the **problem** of predicting the individual motions of a group of celestial objects interacting with each other gravitationally. (Wikipedia)
 - Fourier transform
 - The Fourier Transform is a mathematical tool that breaks a waveform (a function or signal) into an alternate representation, characterized by sine and cosines
 - Sorting
 - Typically, they are computation intensive or data intensive.

- Enterprise applications

- Web-based
- Network-intensive

Regular vs. Irregular grid algorithms

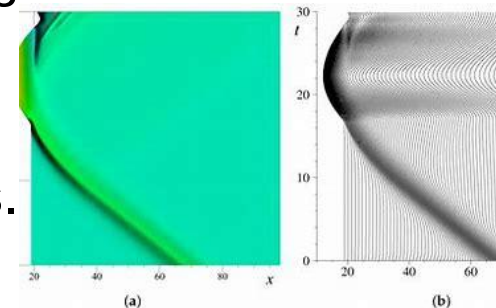
- Regular grid applications

- A regular grid is a structured arrangement of points or cells that are evenly spaced in a multidimensional space.
- The simplicity of the grid structure aids in the implementation of algorithms.
- Eg. Image processing applications (using matrix multiplications)
- NOTE: predictable communication patterns.

$$\begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \begin{bmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \\ b_{3,1} & b_{3,2} \end{bmatrix}$$
$$a_{2,1}b_{1,1} + a_{2,2}b_{2,1} + a_{2,3}b_{3,1}$$

- Irregular grid applications.

- Irregular grids do not maintain consistent spacing between points.
- This can lead to more complex data structures.
- Also, it has to deal with non-uniform distributions.



Parallel Programming

- Parallel programming relies on experience.
- We need to go through the existing parallel programming patterns.
- Design patterns need to be fixed at the initial stage of parallel programming
- There are opportunities to convert sequential programming to parallel programming.
 - Reengineering
- Patterns for Parallel programming – Mattson et al.

Parallel Algorithm Design – Task/Channel Model

- Task/Channel Model

- This model is intended for MIMDs (i.e., multiprocessors and multicomputers) and not for SIMDs.

- A task consists of a

- Program
 - Its local memory
 - Contains the programs instructions and its private data.
 - And, a collection of I/O ports

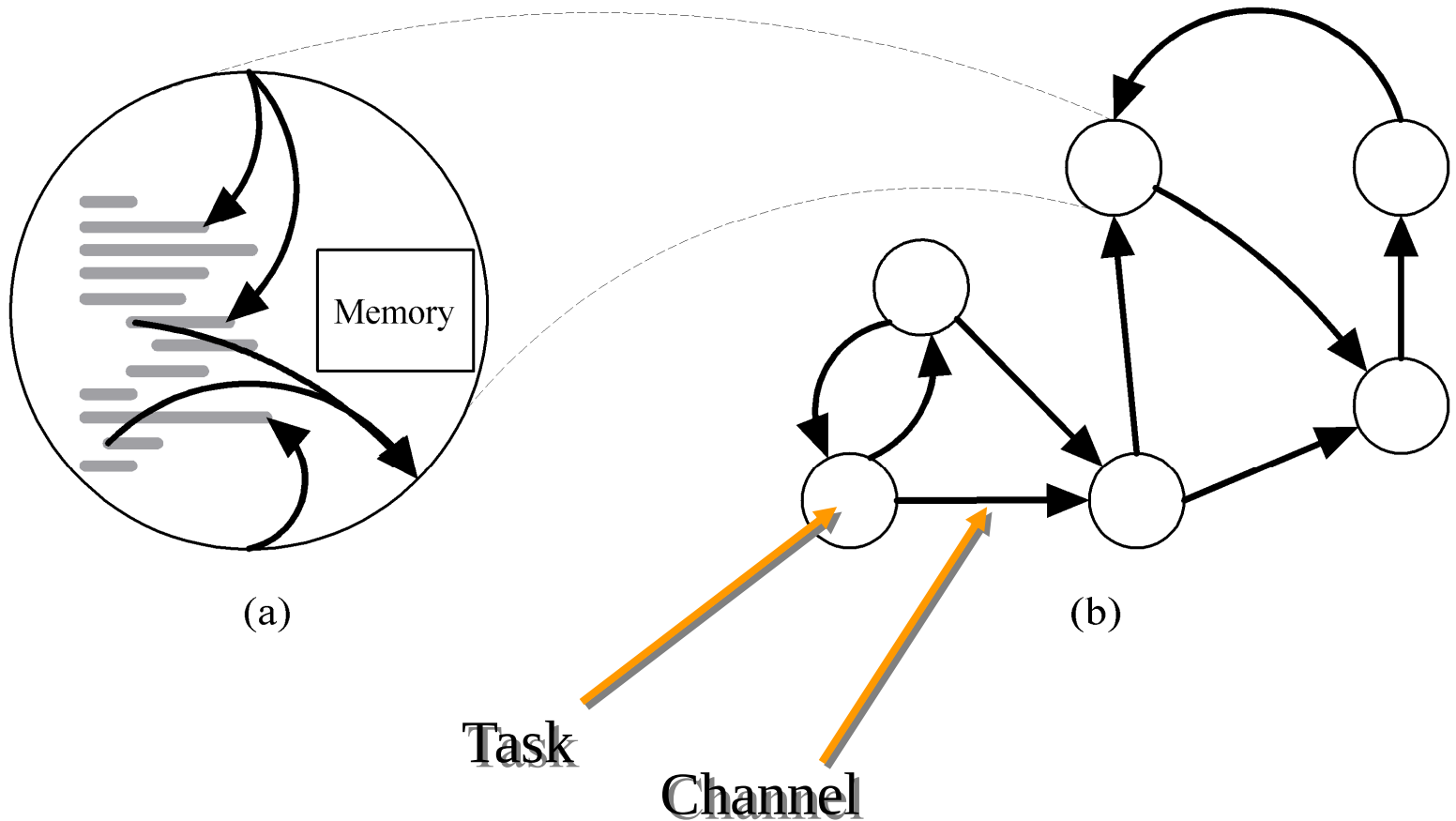
- Tasks interact by sending messages through channels

- A task can send local data values to other tasks via output ports
 - A task can receive data values from other tasks via input ports.

Task/Channel model

- A channel is a **message queue** that **connects** one task's output port with another task's input port.
- Data values appear in input port in the same order in which they are placed in the channel's output queue.
- A task is blocked if a task tries to receive a value at an input port and the value isn't available.
- The blocked task must wait until the value is received.
- **Receiving is a synchronous** operation (blocking is possible) and sending is an asynchronous operation.
- In this model:
 - The **execution time** of a parallel algorithm is the period of time a task is active.
 - The **starting time** of a parallel algorithm is when all parallel tasks simultaneously begin executing.
 - The **finishing time** of a parallel algorithm is when the last task has stopped executing.

Task/Channel Model

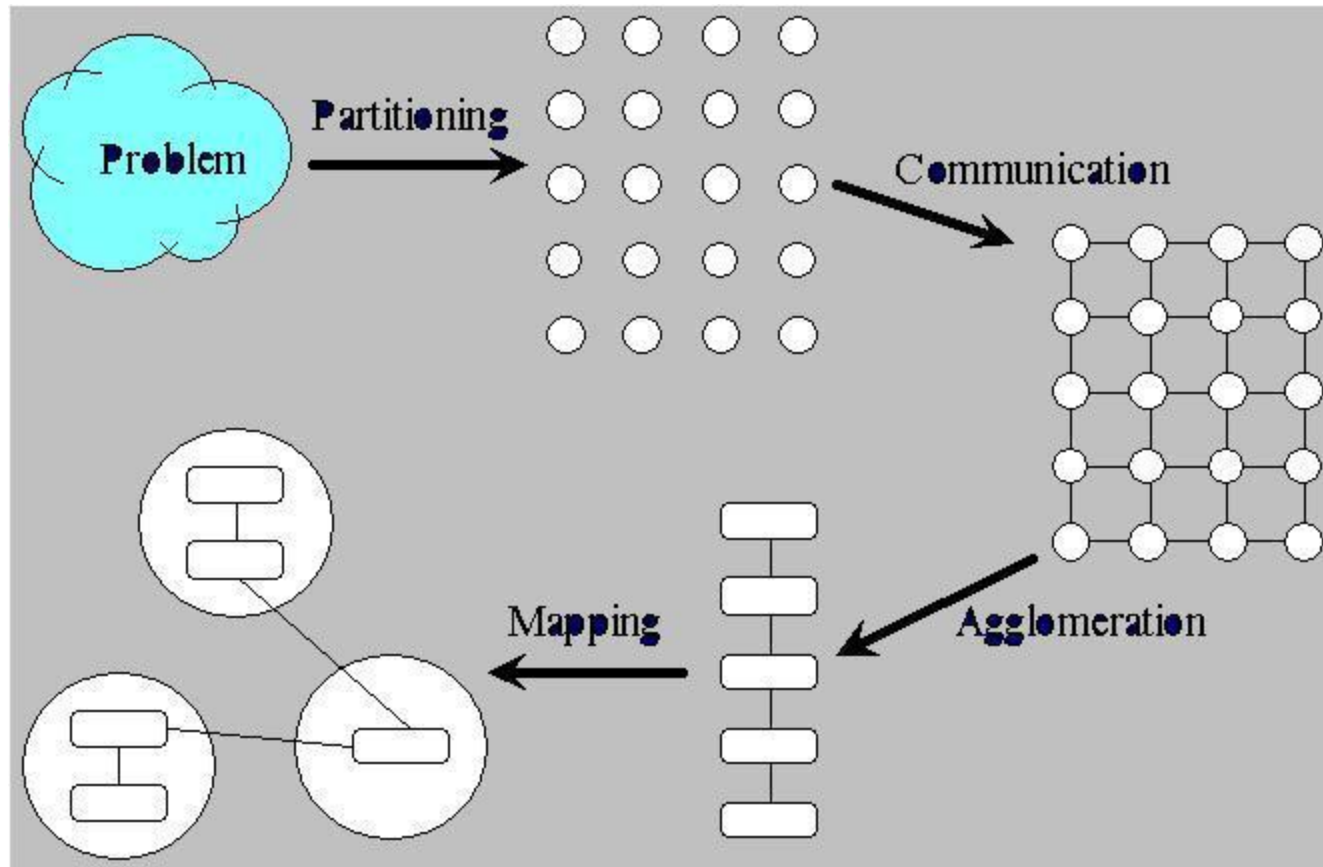


A parallel computation can be viewed as a directed graph

Foster's Design Methodology

- Ian Foster has proposed a 4-step process for designing parallel algorithms for machines that fit the task/channel model.
- Foster's online textbook "**Designing and Building Parallel Programs**" is a useful resource here
- It encourages the development of scalable algorithms by delaying machine-dependent considerations until the later steps.
- The 4 design steps are called:
 - Partitioning
 - Communication
 - Agglomeration
 - Mapping

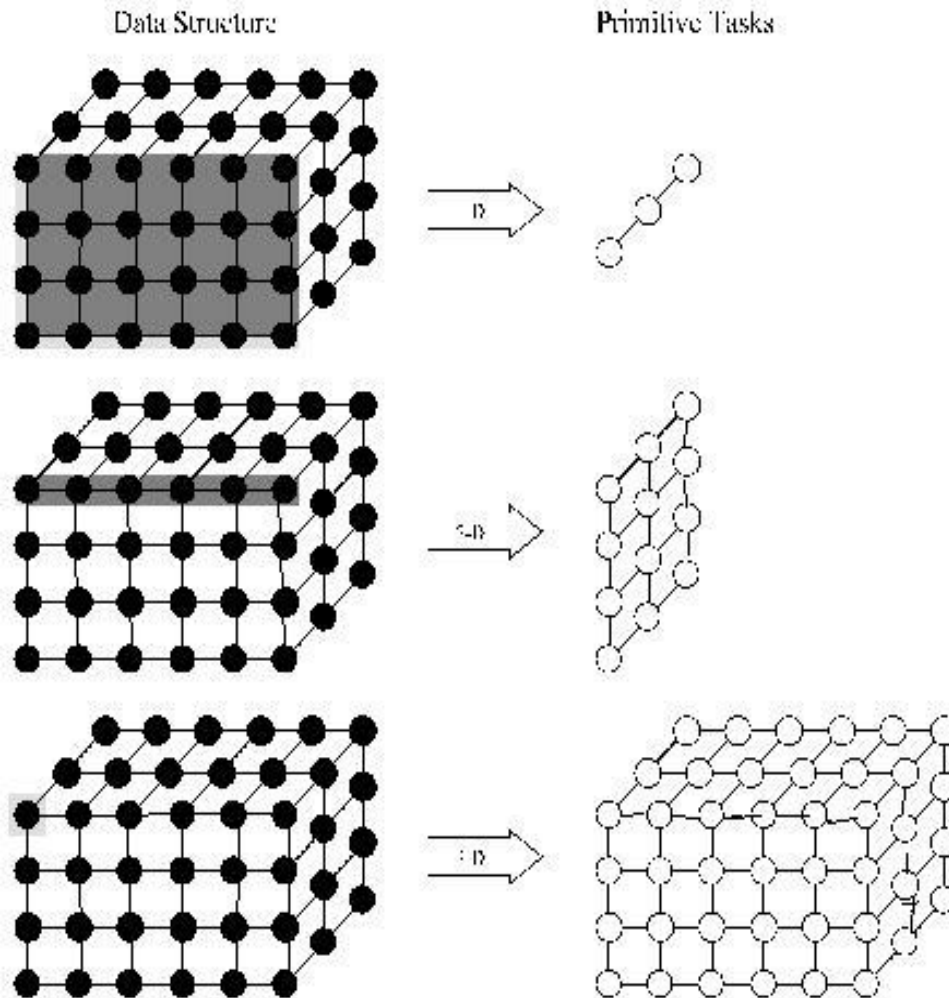
Foster's Methodology



Partitioning

- **Partitioning:** Dividing the computation and data into pieces.
 - Finding potential concurrency.
- **Domain decomposition – one approach**
 - Divide data into pieces
 - Determine how to associate computations with the data
 - Focuses on the largest and most frequently accessed data structure
- **Functional decomposition – another approach**
 - Divide computation into pieces
 - Determine how to associate data with the computations
 - This often yields tasks that can be pipelined.

Example Domain Decompositions

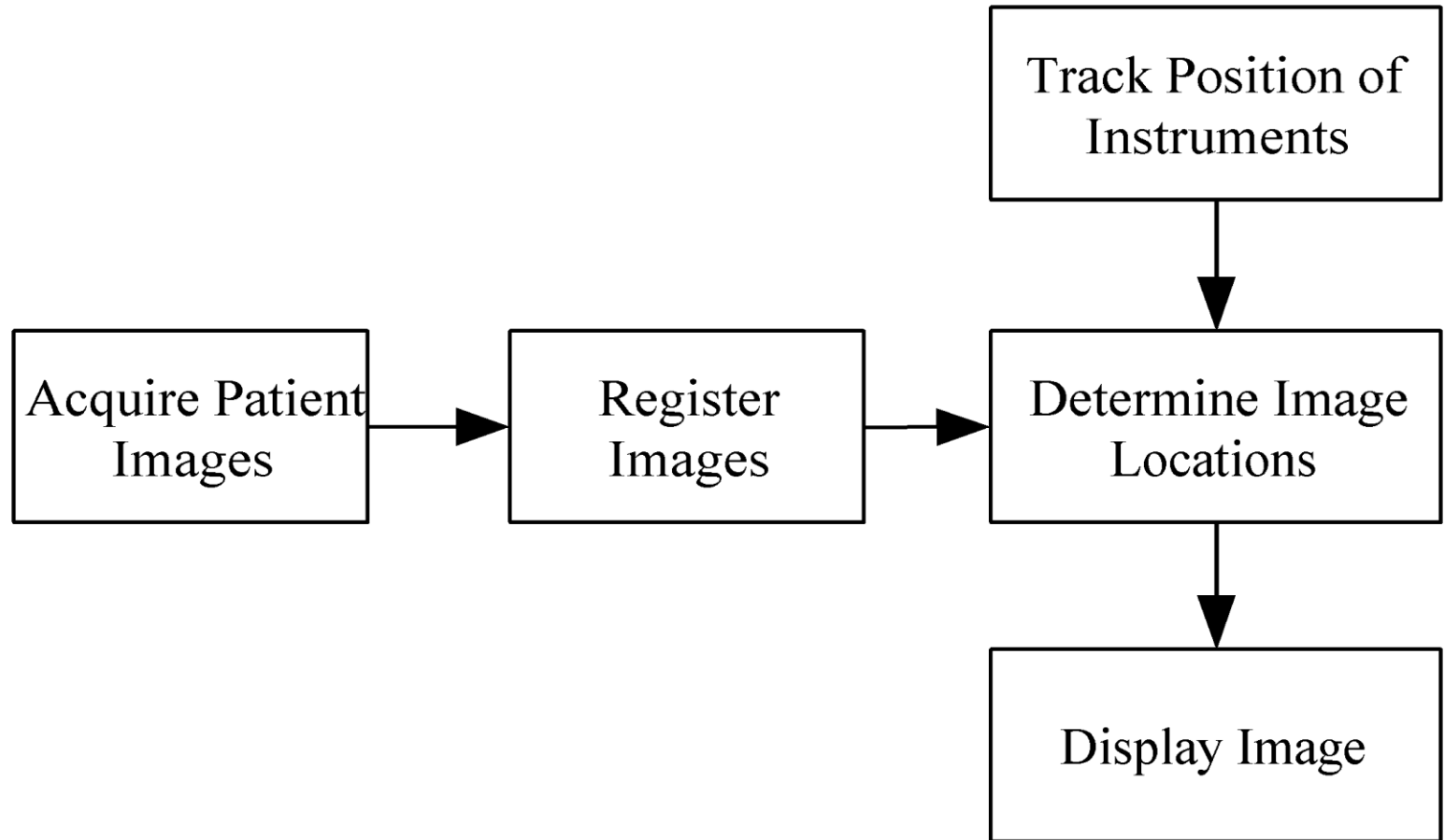


In 1st, each 1D slice is mapped onto one system using 3 processors.

In second, a 2D slice is mapped onto a processor.

In last, a 3D slice is mapped onto a processor

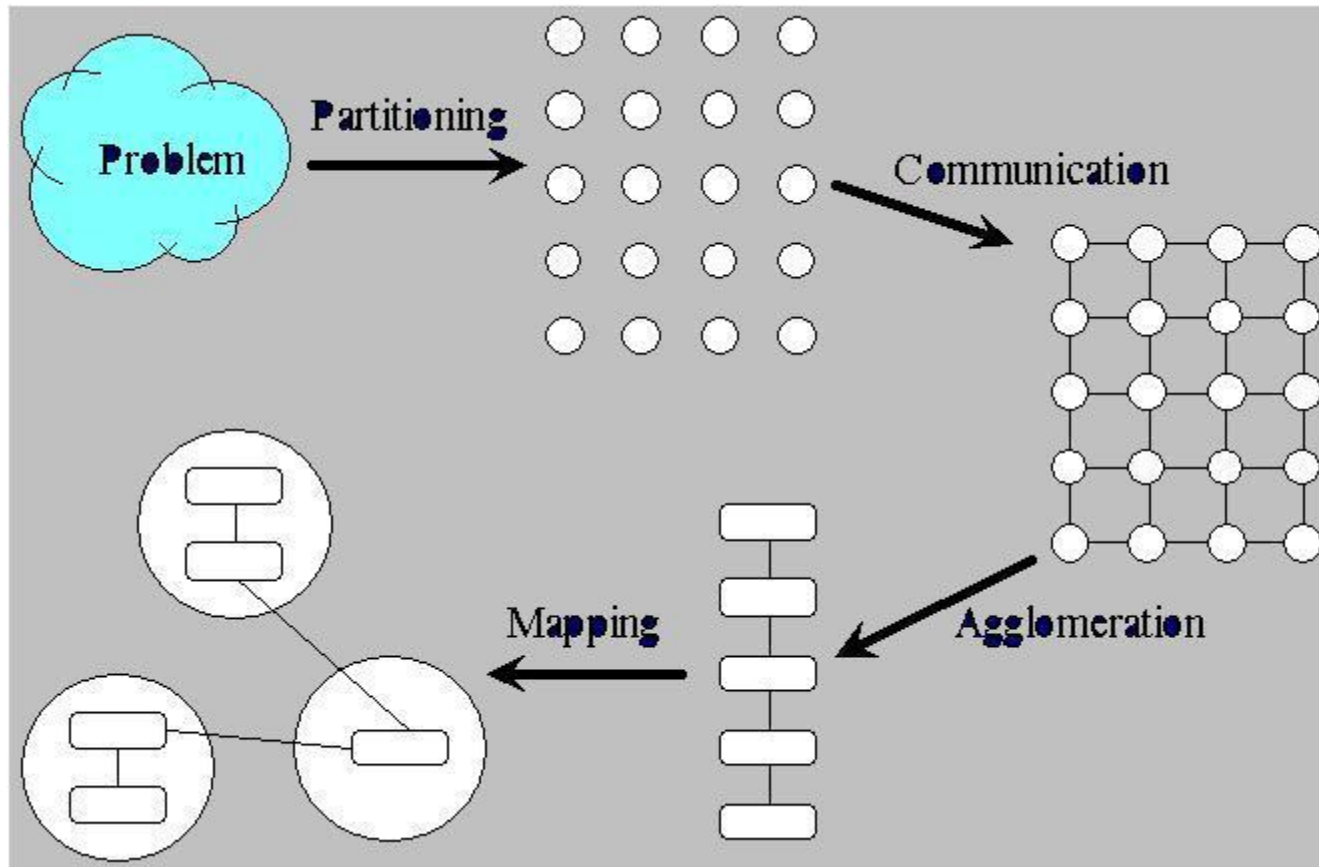
Example Functional Decomposition



Partitioning Checklist

- There are at least **10x more primitive tasks** than processors in a target computer...
- Redundant computations and redundant data storage units are minimized.
- Primitive tasks are roughly the same size so that load balancing is possible.
- The number of tasks is an increasing function of problem size.

Foster's Methodology



Communication

- After we have identified the primitive tasks, the next step is to determine communication pattern between them.
- There are two kinds of communication:
- **Local communication**
 - A task needs values from a small number of other tasks.
 - Create channels illustrating data flow
- **Global communication**
 - A significant number of tasks contribute data to perform a computation
 - Don't create channels for them early in design

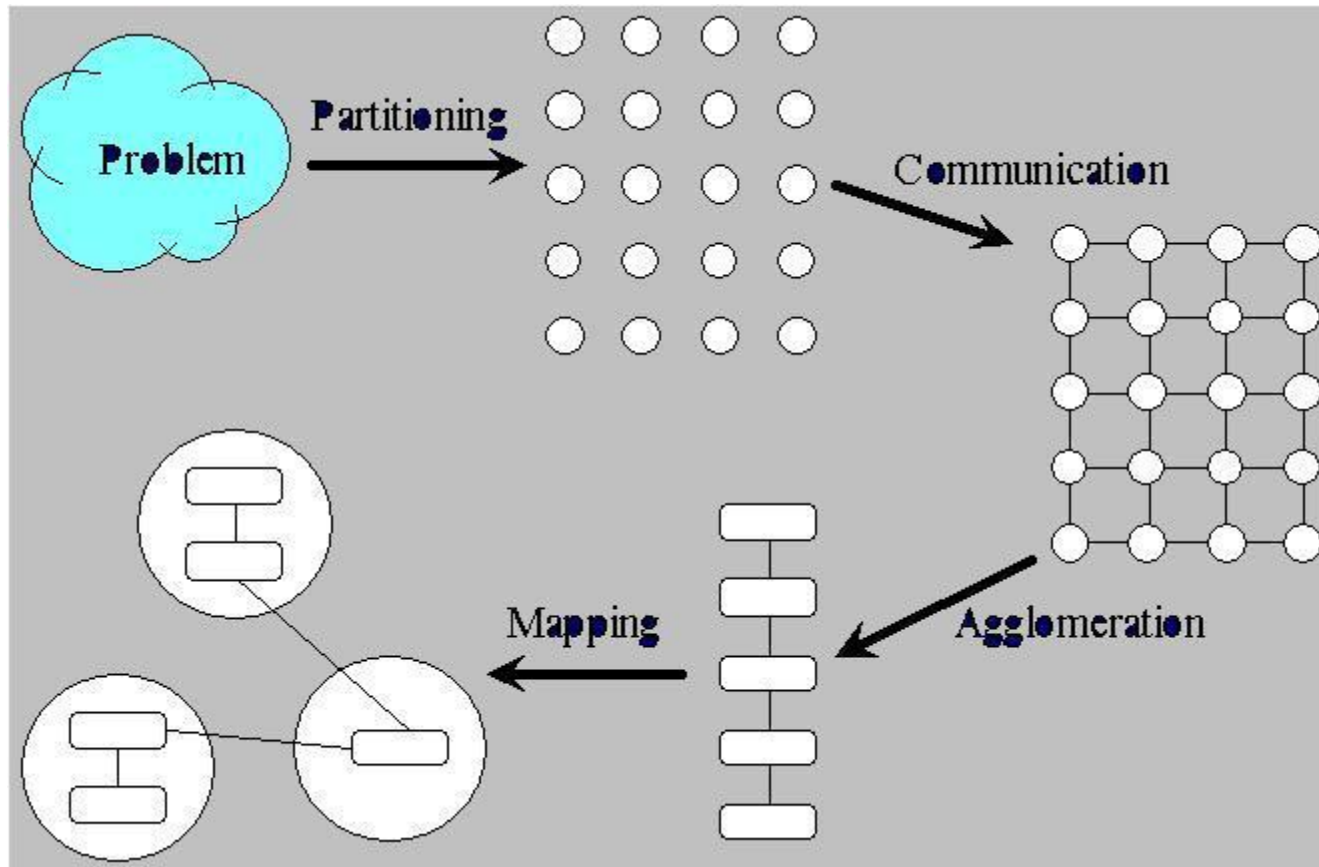
Communication (cont.)

- Communications are part of the parallel computation overhead since it is something sequential algorithms do not have to do.
 - The communication costs could be larger if some (MIMD) processors have to be **synchronized**.
- SIMD algorithms have much smaller communication overhead because
 - Much of the SIMD data movement is between the control unit and the PEs on broadcast/reduction circuits
 - Parallel data movement along with the interconnection network involves lockstep (i.e. synchronously) moves.

Communication Checklist

- Communication operations should be **balanced** among tasks
- Each task communicates with only a small group of neighbors
- Tasks can perform communications concurrently
- Task can perform computations concurrently

Foster's Methodology



Agglomeration and Mapping (Last steps)

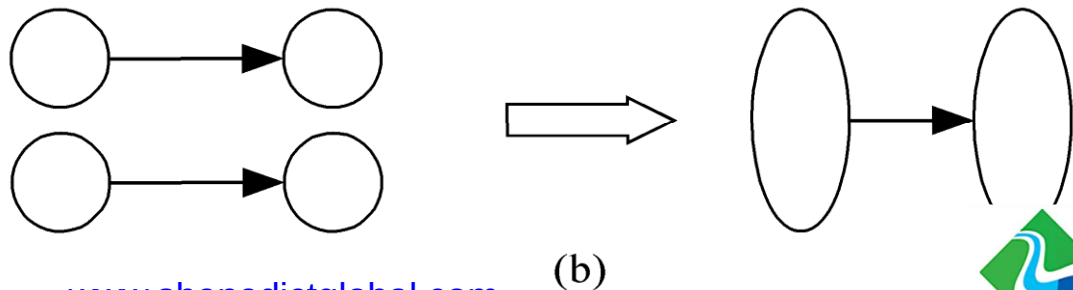
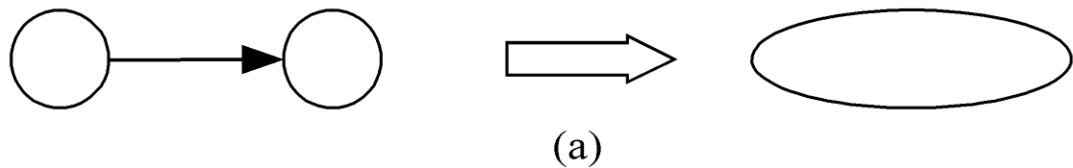
- The first two steps look for parallelism in the problem.
- However, the design obtained at this point probably doesn't map well onto a real machine.
- If the number of tasks greatly exceed the number of processors, the overhead will be strongly affected by how the tasks are assigned to the processors.
- Now we have to decide what type of computer are we targeting ...
 - Is it a centralized multiprocessor or a multicomputer?
 - What communication paths are supported
 - How should we combine tasks in order to map them effectively onto processors?

Agglomeration

- **Agglomeration**: Grouping tasks into larger tasks
- **Goals of Agglomeration**
 - Improve performance
 - Maintain scalability of program
 - Simplify programming – i.e. reduce software engineering costs.
- **In MPI programming, the goals are**
 - to lower communication overhead.
 - often to create one agglomerated task per processor
- **By agglomerating primitive tasks that communicate with each other, communication is eliminated as the needed data is local to a processor.**

Agglomeration – Reduces Commn Overhead

- It can eliminate communication between primitive tasks agglomerated into consolidated task
- It can combine groups of sending and receiving tasks



Agglomeration – Increases Scalability

- Assume

- we are manipulating a 3D matrix of size $8 \times 128 \times 256$ and
- Our target machine is a centralized multiprocessor with 4 CPUs.

- Suppose we agglomerate the 2nd and 3rd dimensions. Can we run on our target machine?

- Yes- because we can have tasks which are each responsible for a $2 \times 128 \times 256$ submatrix.

- Suppose we change to a target machine that is a centralized multiprocessor with 8 CPUs. Can our previous design basically work?

- Yes, because each task could handle a $1 \times 128 \times 256$ matrix.

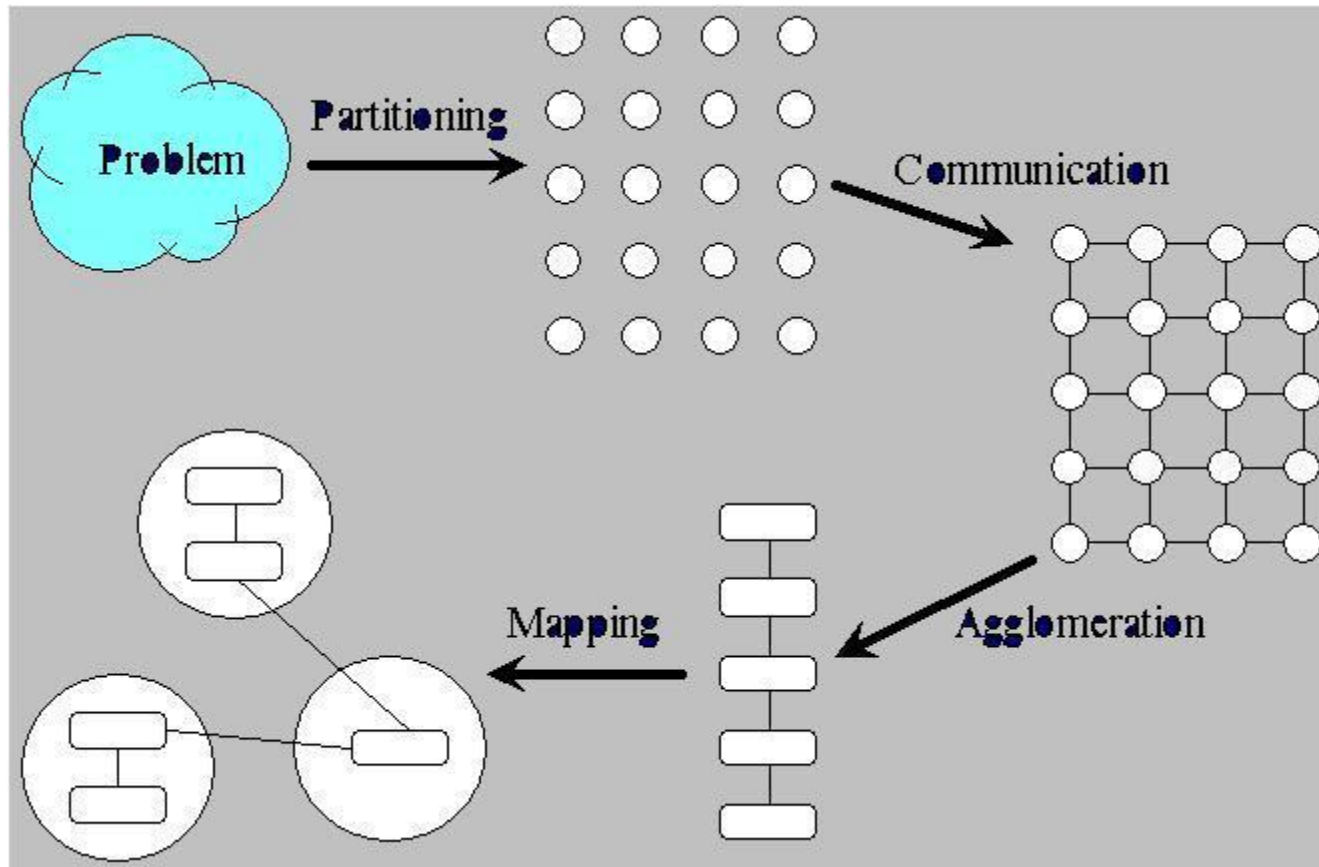
Scalability

- However, what if we go to **more than 8 CPUs**? Would our design fit if we had agglomerated the 2nd and 3rd dimension for the 8 x 128 x 256 matrix?
 - NO.
- This says that the decision to agglomerate the 2nd and 3rd dimension in the long run has the drawback that the **code portability to more CPUs** is impaired.

Agglomeration Checklist

- Locality of parallel algorithm should be increased.
- Replicated computations take less time than communications they replace
- Data replication doesn't affect scalability
- Agglomerated tasks have similar computational and communications costs
- Number of tasks increases with problem size
- Tradeoff between the chosen agglomeration and the code modification costs should be reasonable

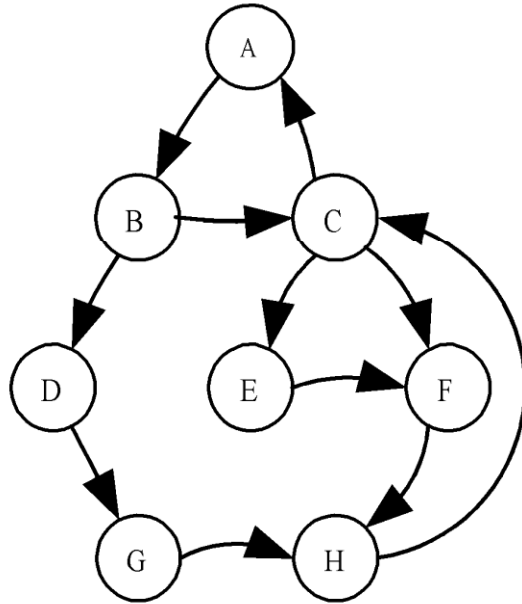
Foster's Methodology



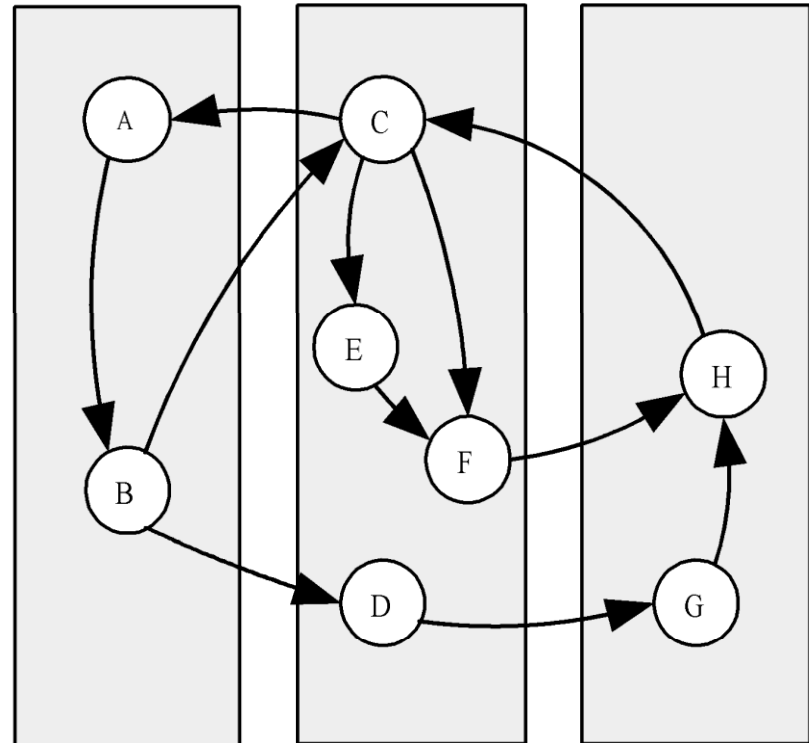
Mapping

- **Mapping:** The process of assigning tasks to processors
- *Centralized multiprocessor:* mapping done by operating system (automatically)
- *Distributed memory system:* mapping done by user
 - We could only do some modifications in such target machines.
- **Goals of mapping**
 - Maximize **processor utilization** – i.e. the average percentage of time the system's processors are actively executing tasks necessary for solving the problem.
 - Minimize interprocessor communication

Mapping Example



(a)

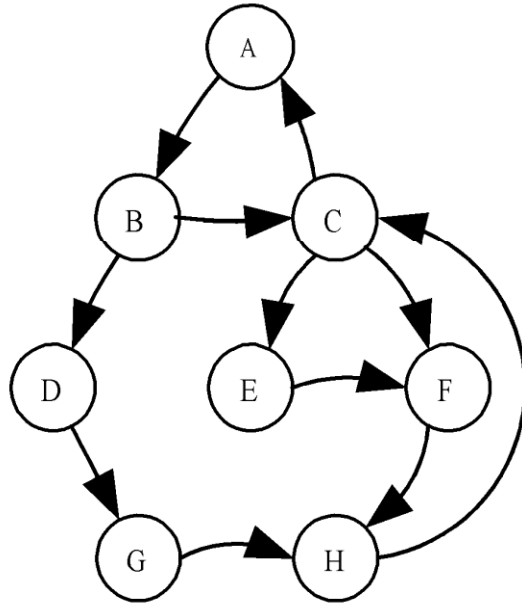


(b)

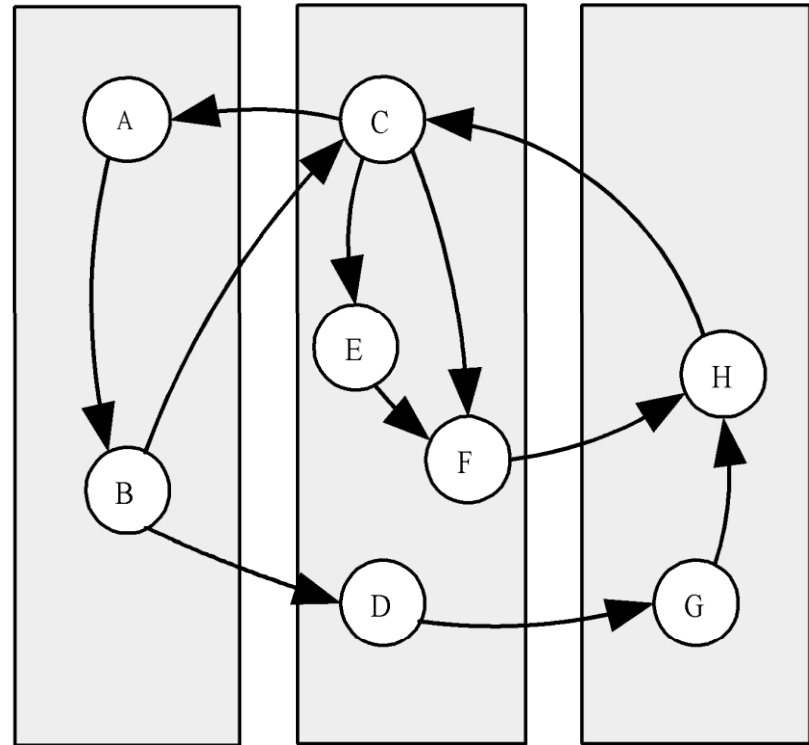
(a) is a task/channel graph showing the needed communications over channels.

(b) shows a possible **mapping of the tasks to 3 processors**²⁸

Mapping Example



(a)



(b)

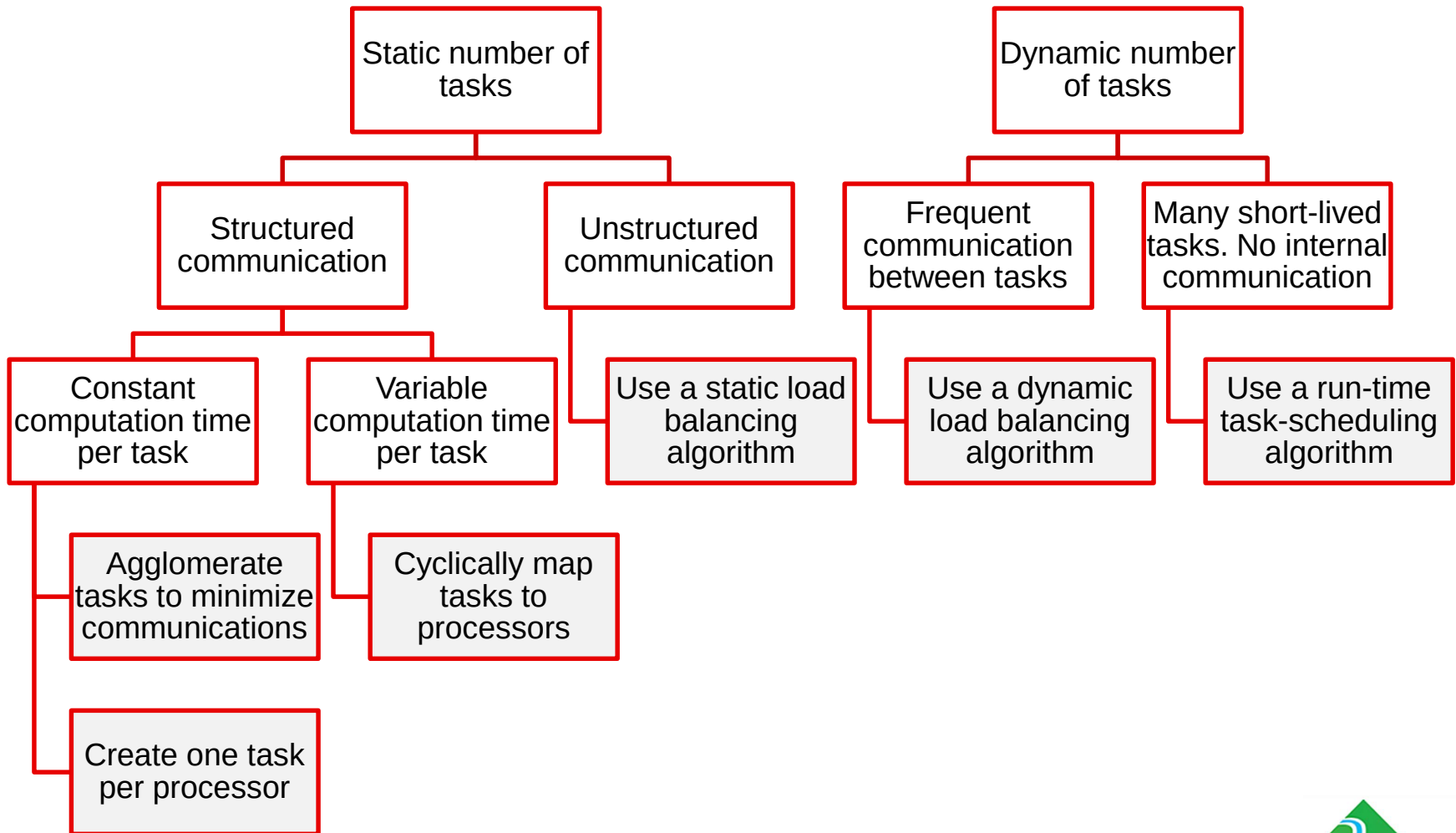
If all tasks require the same amount of time and each CPU has the same capability, this mapping would mean the middle processor will take twice as long as the other two..

Optimal Mapping

- Optimality is with respect to processor utilization and interprocessor communication.
- Finding an optimal mapping is NP-hard.
- Must rely on heuristics applied either manually or by the operating system.
- For example, with p processors and n tasks, putting all tasks on 1 processor makes interprocessor communication zero, but utilization is $1/p$.

A Mapping Decision Tree

Based on characteristics of parallel algorithms

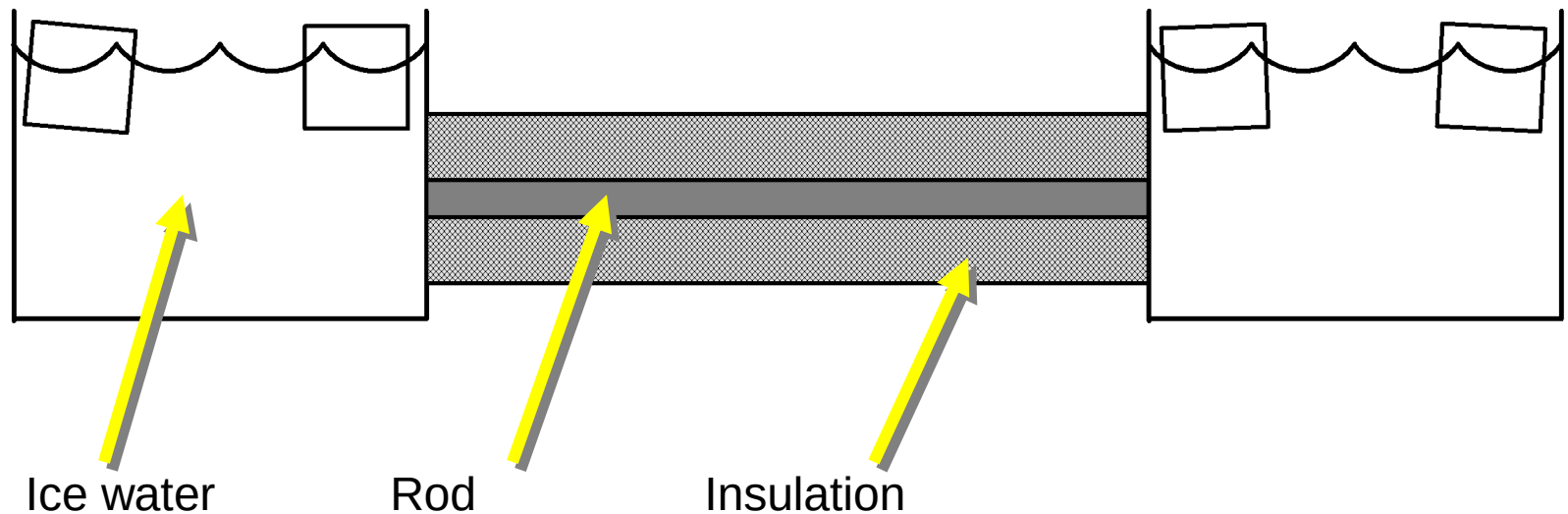


Mapping Checklist

- Consider designs based on one task per processor and multiple tasks per processor.
- Evaluate static and dynamic task allocation to processors.
- If dynamic task allocation was chosen, ensure that the task allocator (i.e., manager) is not a bottleneck to performance.
- If static task allocation was chosen, the ratio of tasks to processors shall have at least 10:1

Boundary Value Problem

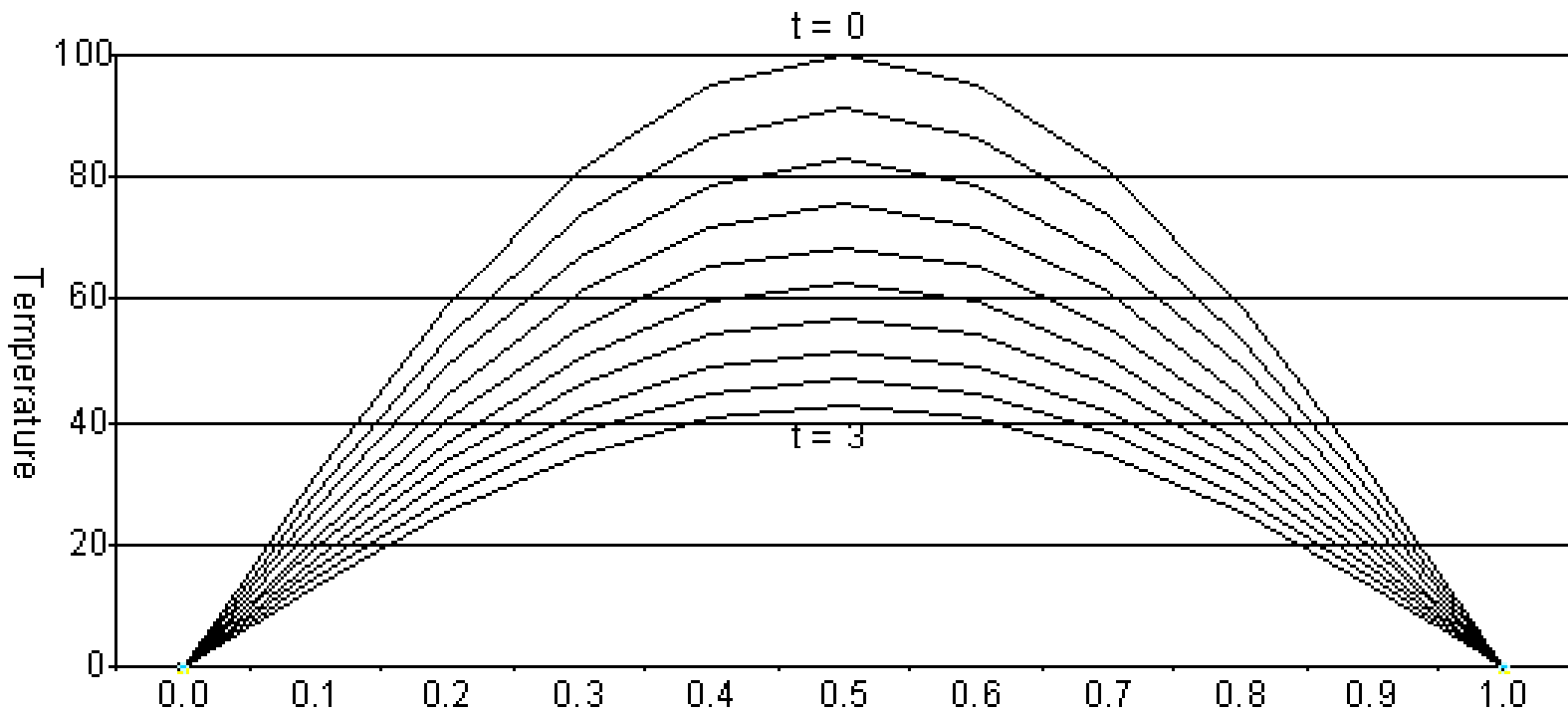
Lets see how to design a parallel algorithm for a Boundary Value Problem...



Assume...

- Rod is made of a uniform material;
- length 1 meter;
- the rod is insulated.
- Both ends have 0 degree temperature.
- The initial temperature u at distance x is $100 \sin(\pi x)$

Rod Cools as Time Progresses



- Over time, the rod gradually cools...
- A partial differential equation models the temperature at any point of the rod at any time.
- The finite difference method is an approach to solve the partial differential eqns.
- Notice: The curve drops as time increases...

Finite Difference Method

- Generally, FDM computes an approximate solution to partial differential equations.
 - It converts Partial Differential Equations to linear equations.
- Here, FDM stores temperatures in a 2D matrix.
- Each row contains temperature distribution of the rod at some point in time.
- Let, rod is divided into n sections of length h ...
- Each row of matrix...
 - each row of matrix has $n+1$ elements
 - Increasing n , reduces the error.
- Let, rod is divided into m discrete entities of height k
- Thus, no. of rows of the matrix...
 - dependent on m ...
 - i.e., $m+1$ rows

Time Dependent problem

$i \rightarrow$ represents a point in the matrix...

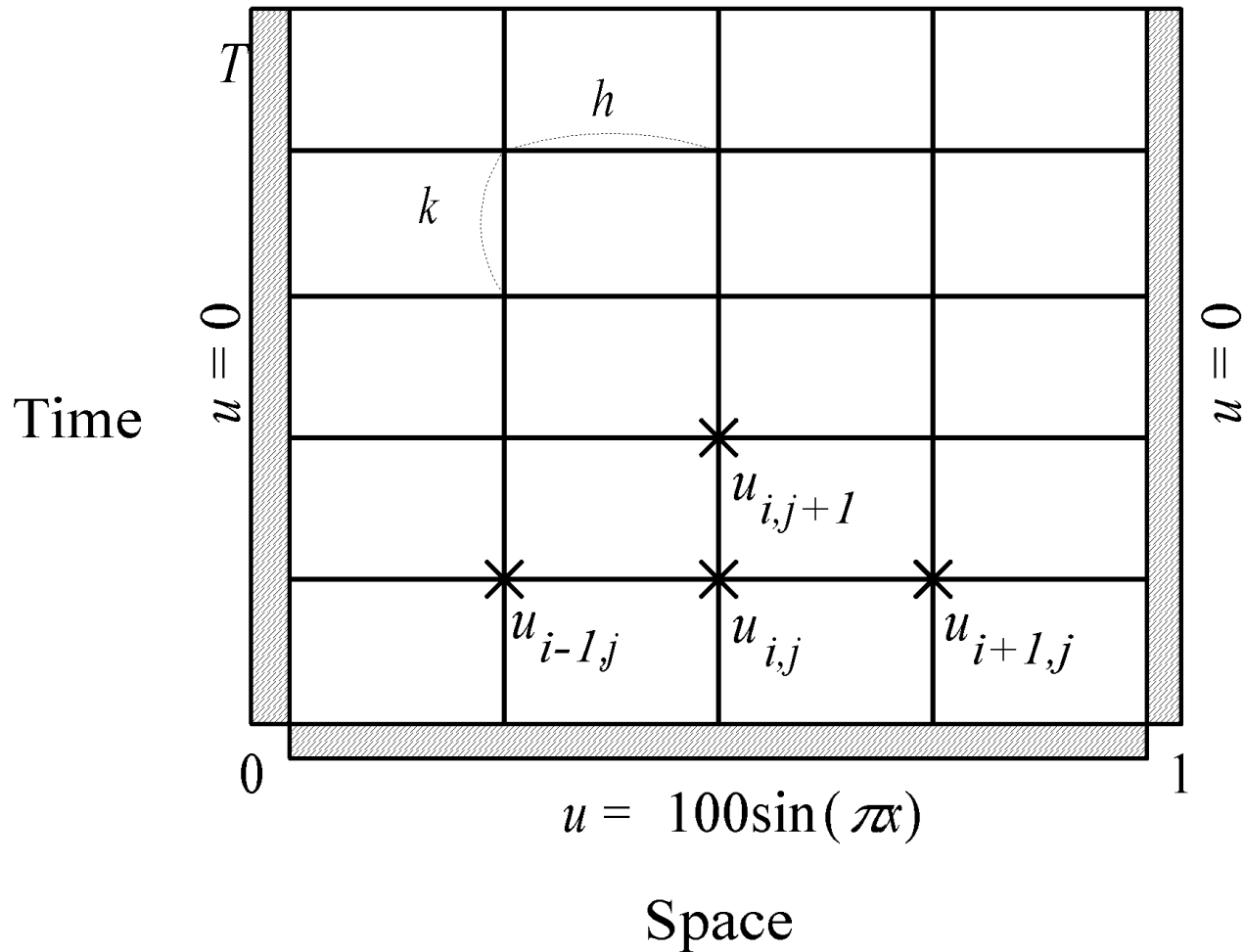
$j \rightarrow$ represents the time level (i.e., height of the rod)

$$r \rightarrow k/h^2$$

- Let $u_{i,j}$ denote the solution on spatial point i and time level j
- Computation formula

$$u_{i,j+1} = r u_{i-1,j} + (1 - 2r)u_{i,j} + r u_{i+1,j}$$

Finite Difference Approximation

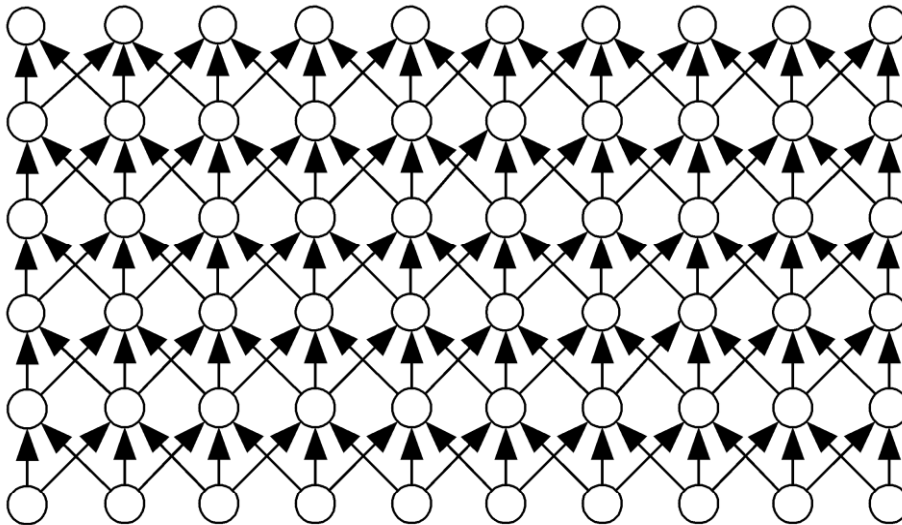


Partitioning

- One data item per grid point
- Associate one primitive task with each grid point
- Two-dimensional domain decomposition

Communication

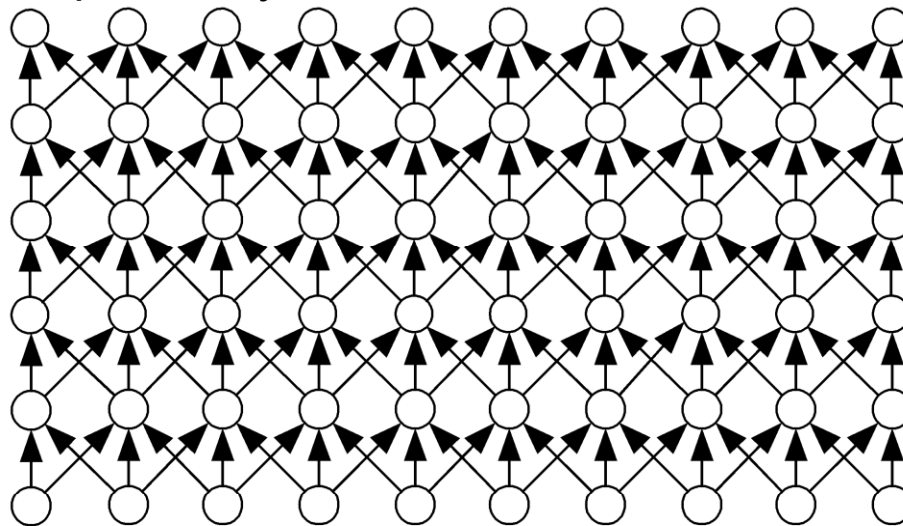
- Identify communication pattern between primitive tasks
- Each interior primitive task has three incoming and three outgoing channels



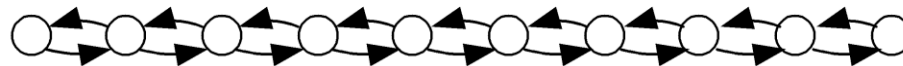
Agglomeration and Mapping

Note: Even if enough processors are available, the primitive tasks could not be executed in parallel (concurrently)...

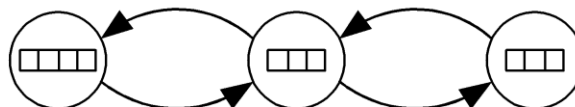
Why? Dependency...



(a)



(b)



(c)

Agglomeration

N-body Simulation – Use Case

- N-body Simulation

- A physics problem...
- It is a problem of predicting the individual motions of a group of celestial objects interacting with each other gravitationally (Wikipedia).
- Here, every particle exerts a gravitational pull on every other particle that are having varying mass.

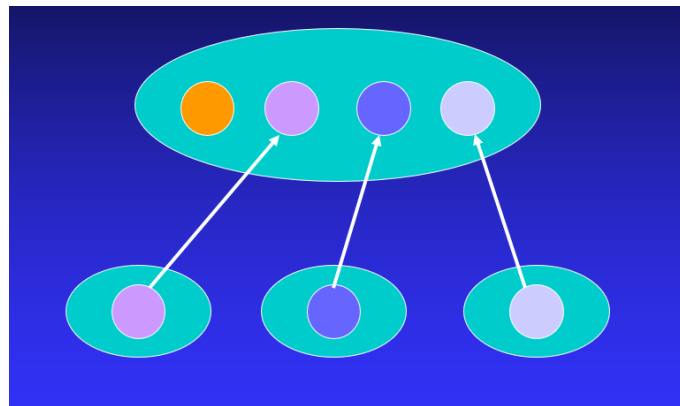
- Partitioning (Identifying tasks -- subproblems)

- One task per particle...
- How does the particle move? We want to find the new location of the particle...
- The particles are moving in the free space...
- It could hit each other??
- Each task (i.e., particle) shall have a new position and velocity information...

N-body Simulation

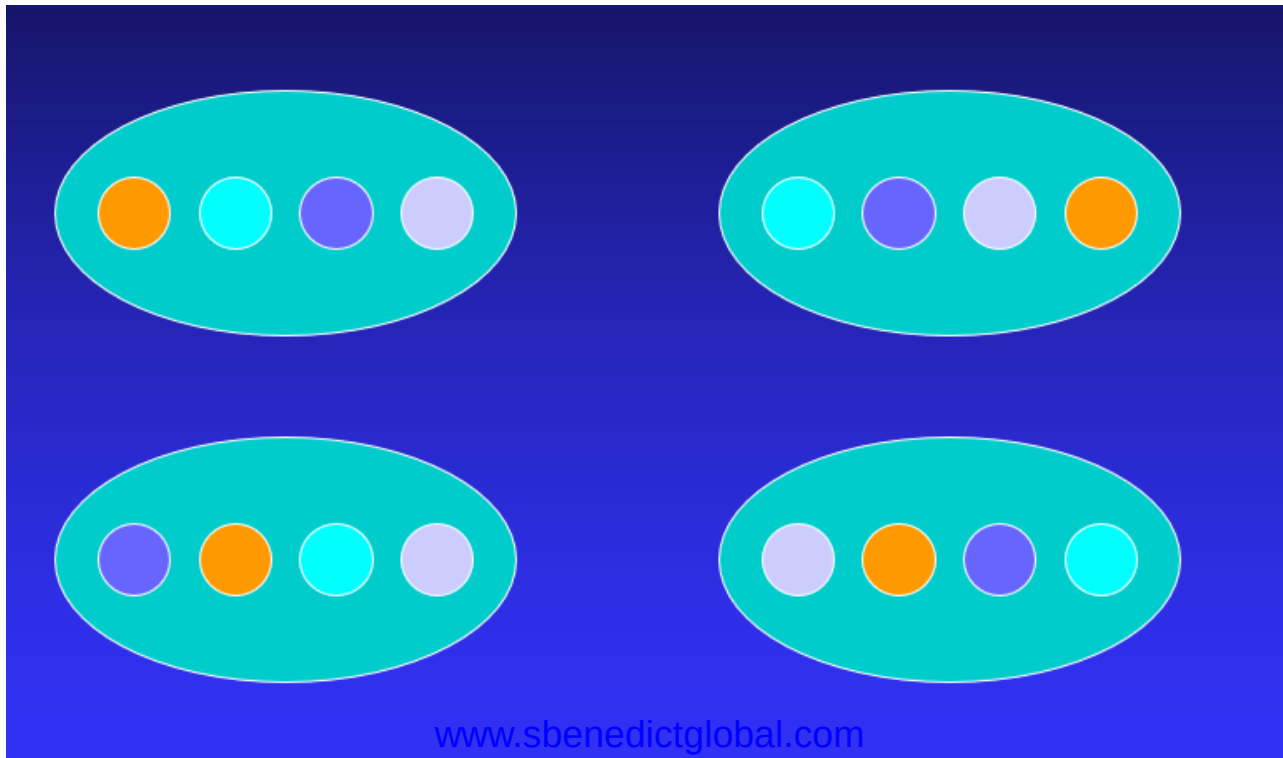
- Communication

- To compute the newer location, it may need to get the location information of all other particles...
- A sort of **gather** operation.
 - (i.e. a global communication)
 - (i.e., data are collected from multiple tasks to a single task).
 - It is a concatenation of data items.



N-body Simulation

- An **all-gather** operation could also be performed
 - (i.e., a global communication similar to gather operation)
 - It attempts to concatenate a set of data items on all tasks.
 - Difference is that at the end of the communication, every task has a **copy of the entire data**.



Agglomeration and Mapping

- We could agglomerate n number of particles per processor ...
- And, perform “all-gather” operation at the end of computations.

Dependence → Leading to Hazards

Dependences

Data Dependence →
data hazards

```
a=b+c;  
d=a+1;
```

Previous o/p becomes an i/p

Name Dependence
→ Structural hazards

```
a=b+c;  
b=d+1;
```

Previous name is used.

Control dependence
→ Control hazards

```
If(x>0) y=y+1;
```

Instruction depends on branch conditions

Impacts of dependencies

- Data dependency or any dependency conveys:
 - the possibility of a hazard,
 - the order in which results must be calculated, and
 - an upper bound on how much parallelism can be exploited.

Dependence → Leading to Hazards

- Data Hazard

- The output of the immediate previous instructions are required as input to the pipelined instruction.

- Structural Hazard

- Hardware cannot support the combination of instructions that we want to execute in the same clock cycle.
- For e.g., hardware non-availability or hardware malfunctioned.
- For e.g., if an MIPS computer is designed with only one memory unit (i.e., IM and DM are using the same memory).

- Control Hazard

- Delay happening due to decision making processes.
- For e.g., branching to a loop... (Should I really branch? – some prediction or testing is required – i.e., we spent some time.)

- One Soln: Wait till the previous instruction or decision making steps is complete.

Pipelined Architectures

- In a pipelined processor, operations are divided into stages that are executed in parallel.
- For instruction-level pipelined processors
 - Fetch
 - Decode
 - Execution
 - Memory access
 - Write results

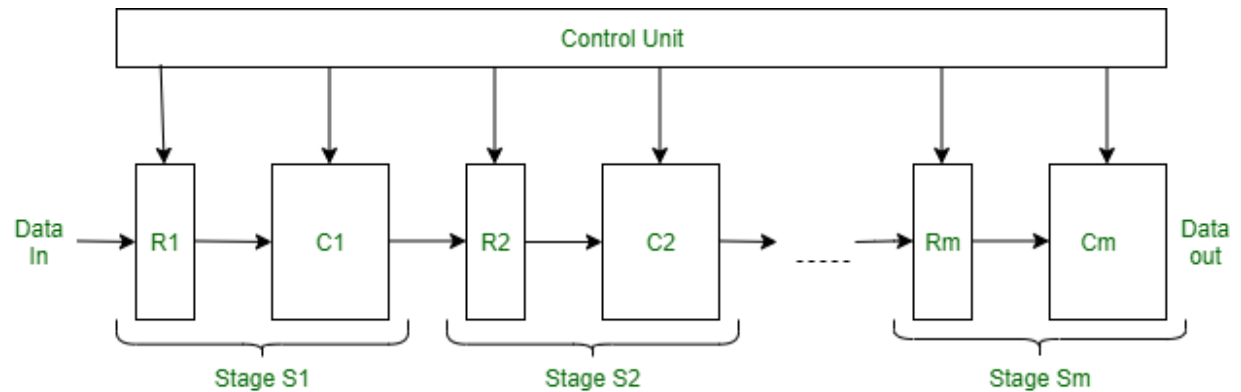
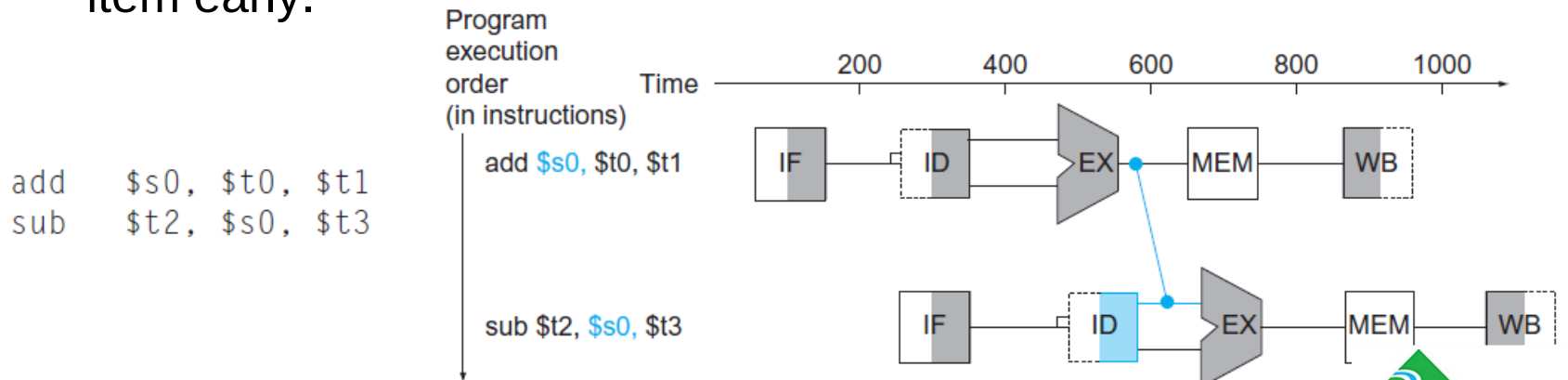


Figure - Structure of a Pipeline Processor

Problems and Solutions – Data Hazard

• Soln1: Forwarding or Bypassing

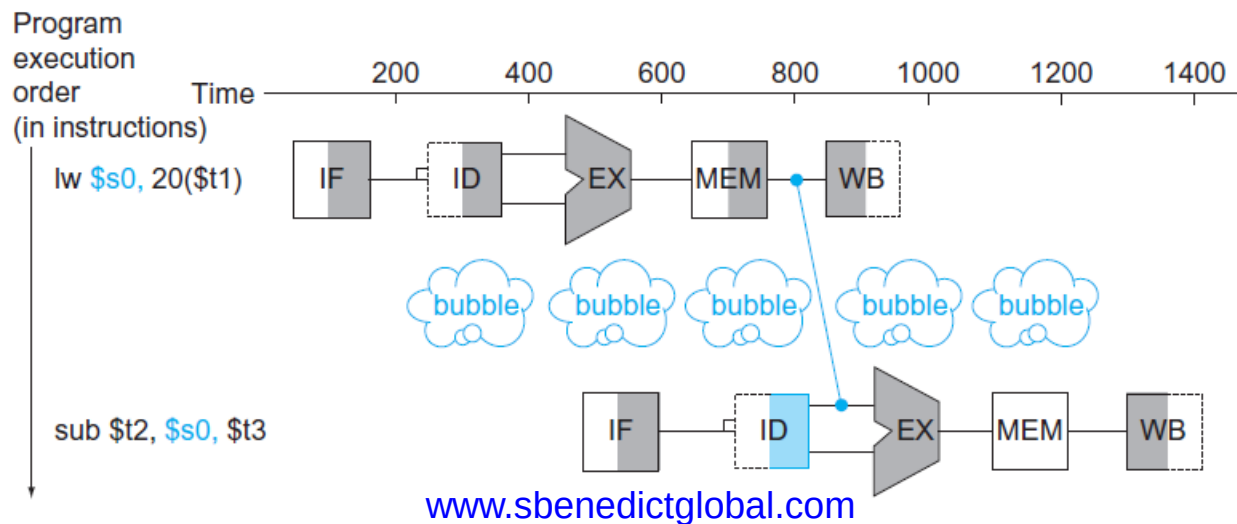
- It presents the result of the instruction at the execution stage to the pipelined immediate instruction.
- i.e., the results are kept for *early access* without reaching the last stage.
- i.e., Don't wait till the end of 5th stage of the previous instruction for the data to be ready...
- Typically, **specific hardware (forwarding unit and Hazard detection unit)** will be utilized to retrieve the missing data item early.



Problems and Solutions – Data Hazard

- Soln2: Bubbling or Pipeline stall

- In some situations, the previous approach could not be adopted.
- E.g., lw and Sub instructions.
- NOTE: results will be available only after MEM stage...
- NOTE: Time cannot be earlier... Hence, we need to delay the time using specific hardware (termed as bubble).
- Bubble – stalls the execution for some time.



Problems and Solutions – Data Hazard

- Soln3: Reordering the instructions
 - Usually using compilers.

Control Hazard

- Solution: Branch prediction

- Static approach vs. Dynamic approach.
- Predict with specific accuracy (i.e., 90% accuracy).
- Branch predictions are handled whenever branch instructions are encountered in the CPU.
- A simple prediction method:
 - Always predict that the branching is untaken.
 - When you're right, the pipeline proceeds at FULL SPEED.
 - When you're wrong, stalls are experienced.

Performance Study – Pipelined Processors

- Speedup (S)

= Non-Pipelined Execution Time / Pipelined Execution Time

- Efficiency

= Speedup / No.of Stages in pipelined arch.

- Throughput

= No.of instructions executed / Total Time taken

Practice Problems – Ideal case

- A non-pipelined processor executes an instruction in 5 clock cycles (IF, ID, EX, MEM, WB). Assume each stage takes 10 ns. Find the execution time for 100 instructions in the non-pipelined processor.

ANS: Each instruction = $5 \times 10\text{ns} = 50\text{ ns}$

ANS: Hence, $100 \Rightarrow 5000\text{ ns}$.

Practice Problems – Ideal Case

- A non-pipelined processor executes an instruction in 5 clock cycles (IF, ID, EX, MEM, WB). Assume each stage takes 10 ns. Find the execution time for 100 instructions in a pipelined processor.
- NOTE: Pipelined processors depend on cycles.
- Cycle time = Delay offered by one stage including the delay due to its register.

ANS: Assume an ideal pipelined processor

ANS: Find no.of cycles for k stage pipeline processor:

$$\begin{aligned}\text{Cycles} &= k + (n - 1) \quad \leftarrow \text{FORMULA} \\ &= 5 + 99 \\ &= 104\end{aligned}$$

ANS: Hence, $104 \times 10\text{ns} \Rightarrow 1040 \text{ ns}$

Practice Problems – Ideal Case

- A non-pipelined processor executes an instruction in 5 clock cycles (IF, ID, EX, MEM, WB). Assume each stage takes 10 ns. Find the speedup
- Speedup = 5000/1040

ANS: Hence, 4.81 ns

Practice Problems – In Real-world

- NOTE: pipelined processors can encounter delays between stages.
- Hence, Cycle time = Maximum delay offered by any stage including the delay due to its register.
- Problem: A 4-stage pipeline has stage delays: 8 ns, 10 ns, 6 ns, 7 ns. What is the **clock cycle time** of the pipeline?
- ANS: max stage delay = 10 ns

Practice Problems – In Real-world

- Problem: A 4-stage pipeline has stage delays: 8 ns, 10 ns, 6 ns, 7 ns. Find the Throughput.
- ANS:
 - $\text{Throughput} = \text{No. of instructions executed} / \text{Total Time taken}$
 - $\text{Throughput} = 1/10 \text{ ns} = 0.1/\text{ns} = 100 \text{ MIPS}$.